UNIVERSITY OF PETROLEUM AND ENERGY STUDIES
DEHRADUN

Applied DevOps

Experiment 9

MTECH - School of Computer Science
CYBER SECURITY AND FORENSICS

Submitted To: Dr. KrishnaRaj

Name: Jigesh Sheoran
SAP ID: 590025428

Objective — To compare infrastructure isolation and automation between container-based environments using Docker and virtual machines using VirtualBox by deploying and analyzing a simple web application.

Tools and Environment :

Development Environment

- Operating System: macOS (MacBook Air)
  IDE: Visual Studio Code
- Terminal: zsh shell

Version Control

- Git: Used for tracking source code changes and managing versions
- GitHub: Cloud-based repository hosting platform
  Authentication Method: SSH key-based authentication.

Task to be Performed

Step 1: Create project

```
Last login: Fri Apr 24 22:05:07 on ttys000
sheoraninfosec@Jigeshs-MacBook-Air ~ % mkdir exp9
sheoraninfosec@Jigeshs-MacBook-Air ~ % cd exp9
sheoraninfosec@Jigeshs-MacBook-Air exp9 %
```

Step 2: create web app

```
  UW PICO 5.09                      File: index.html                      Modified

<h1>Running inside Docker Container</h1>
<p>This demonstrates container-based isolation.</p>
```

Step 3: Dockerfile

```
  UW PICO 5.09                      File: Dockerfile                      Modified

FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Step 4: Build & Run

```
exp9 — docker-buildx ◂ docker build -t exp9 . — 80×24

sheoraninfosec@Jigeshs-MacBook-Air exp9 % docker build -t exp9 .
 [+] Building 21.6s (5/7)                                  docker:desktop-linux
  => [internal] load build definition from Dockerfile                      0.0s
  => => transferring dockerfile: 114B                                      0.0s
  => [internal] load metadata for docker.io/library/nginx:latest           5.5s
  => [auth] library/nginx:pull token for registry-1.docker.io              0.0s
  => [internal] load .dockerignore                                         0.0s
  => => transferring context: 2B                                           0.0s
  => [internal] load build context                                         0.0s
  => => transferring context: 130B                                         0.0s
  => [1/2] FROM docker.io/library/nginx:latest@sha256:6e23479198b998e5e25  16.1s
  => => resolve docker.io/library/nginx:latest@sha256:6e23479198b998e5e259  0.0s
  => => sha256:91409cde2cec3cd84a28ca803ae3d24bf1578bcdb38 1.40kB / 1.40kB  0.8s
  => => sha256:405d5794f35e5f3b100155d8204811fa8a3d876c145 1.21kB / 1.21kB  1.3s
  => => sha256:0c0e4c0190085dbb13856d7ba75a57cae147d540d83e892 404B / 404B  1.6s
  => => sha256:b420423b8e46de20c9d414c28603a7dabc4b790dac6f030 955B / 955B  1.6s
  => => sha256:1b39715bb7021ba05df1e673b866343e1fe6b4245d5bbd6 628B / 628B  0.9s
  => => sha256:7ebd5beed79e7f07fdd0a1dcba0b119dbe2080590 31.14MB / 31.14MB  8.2s
  => => sha256:e4fb5f1cd4d4ee56da574ef5ed88a5c74f100ba9 30.14MB / 30.14MB  14.4s
```

```
[sheoraninfosec@Jigeshs-MacBook-Air exp9 % docker run -d -p 8082:80 exp9        ]
9b838e8bf09a0bc513aca78dc947ba1f6a4ce96f422207f4d76c96e18d8ac699
sheoraninfosec@Jigeshs-MacBook-Air exp9 %
```

Step 5: Verify

Running inside Docker Container

This demonstrates container-based isolation.

Observation

- Application runs instantly
- Lightweight environment
- No OS installation required

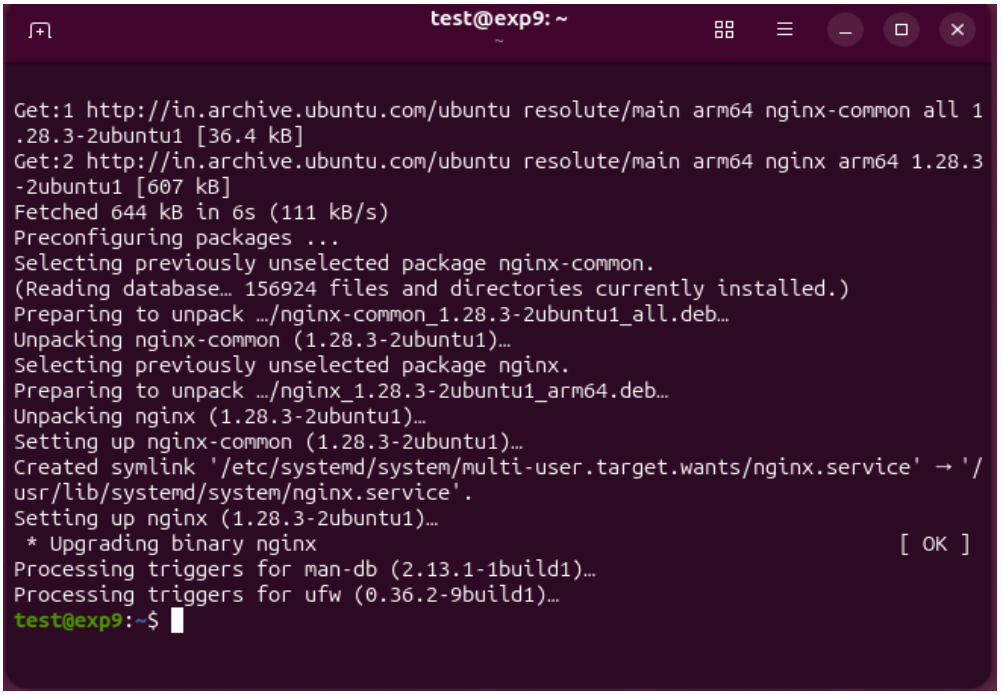
Task 2: Virtual Machine Setup

Step 1: Install Virtual Box

Step 2: Create VM

Step 3: Inside VM:

Install nginx server using **sudo apt install nginx -y**



```
test@exp9:~$ sudo nano /var/www/html/index.html
```

```
 GNU nano 8.7.1                /var/www/html/index.html *
<h1> Running inside VM </h1>
<p> Demostrating VM isolation <p>
```

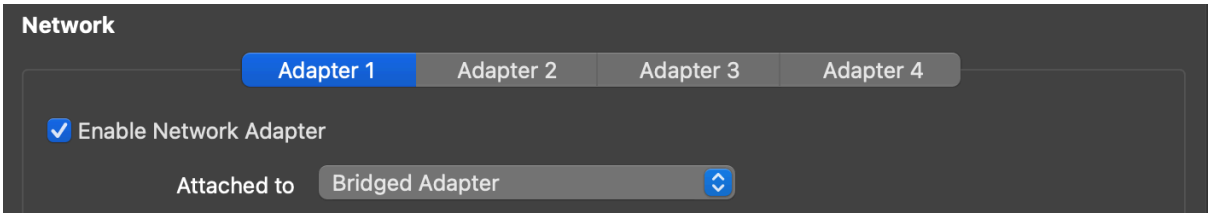
Step 4: Access via IP

inet 10.0.2.15

```
test@exp9:~$ sudo nano /var/www/html/index.html
test@exp9:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
    link/ether 08:00:27:5a:72:a1 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s8
       valid_lft 85823sec preferred_lft 85823sec
    inet6 fd17:625c:f037:2:a00:27ff:fe5a:72a1/64 scope global dynamic noprefixro
ute
       valid_lft 86186sec preferred_lft 14186sec
    inet6 fe80::a00:27ff:fe5a:72a1/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
test@exp9:~$
```

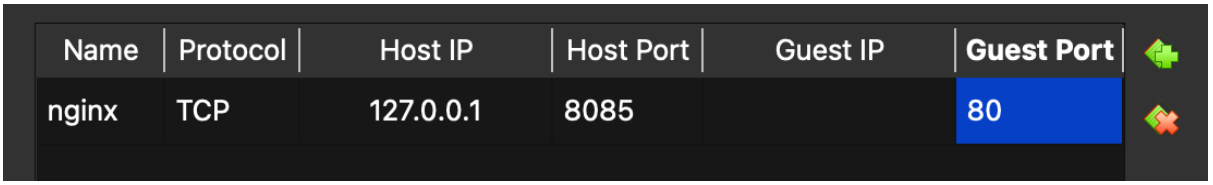
But this ip is local and is not accessible on mac. So we need to reconfigure the Ubuntu VM.

Change attached Network adapter to Bridge Adapter

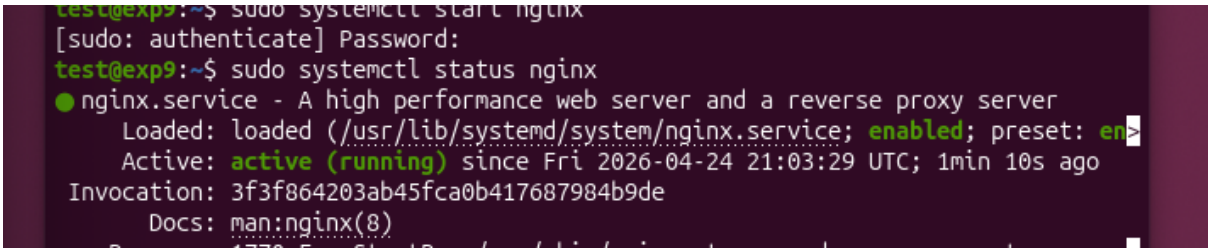



```
oup default qlen 1000
    link/ether 08:00:27:5a:72:a1 brd ff:ff:ff:ff:ff:ff
    inet 172.20.10.2/28 brd 172.20.10.15 scope global dynamic noprefixroute
s8
```

Still not being able to connect to mac so now we will try to use another approach i.e **NAT + Port Forwarding**



Save the config and restart ubuntu.



Running inside VM

Demostrating VM isolation

Successfully connect to mac.

Observation

- Slower setup
- Full OS running
- Higher resource usage

Task 3: Analyze Infrastructure Isolation

1. Observation and Comparison

Process-Level Isolation in Containers

- Containers use process-level isolation.
- All containers share the host operating system kernel.
- Isolation is achieved using:
  - Namespaces (separate processes, network, filesystem)
  - Control groups (resource limits)
- Containers are lightweight and run as isolated processes.

OS-Level Isolation in Virtual Machines

- Virtual Machines provide full OS-level isolation.
- Each VM runs its own operating system on virtualized hardware.
- A hypervisor (like VirtualBox) manages multiple VMs.
- VMs are completely independent from the host system.

2. Differences Observed

Resource Usage

- Containers:
  - Low resource consumption
  - Share host OS → no extra OS overhead
  - Efficient memory and CPU usage
- Virtual Machines:
  - High resource consumption
  - Each VM runs a full OS
  - Requires dedicated RAM, CPU, and storage

Startup Time

- Containers:
  - Start in seconds
  - No OS boot required
- Virtual Machines:
  - Take minutes to start
  - Full OS boot process involved

Environment Separation

- Containers:
  - Moderate isolation
  - Share kernel → less secure than VMs
  - Suitable for application-level isolation
- Virtual Machines:
  - Strong isolation
  - Completely separate OS and environment
  - More secure and suitable for critical workloads

Task 4: Container Automation

Step 1: Remove Container

```
[sheoraninfosec@Jigeshs-MacBook-Air exp9 % docker rm -f 9b838e8bf09a0bc513aca78dc]
 947ba1f6a4ce96f422207f4d76c96e18d8ac699
 9b838e8bf09a0bc513aca78dc947ba1f6a4ce96f422207f4d76c96e18d8ac699
 sheoraninfosec@Jigeshs-MacBook-Air exp9 %
```

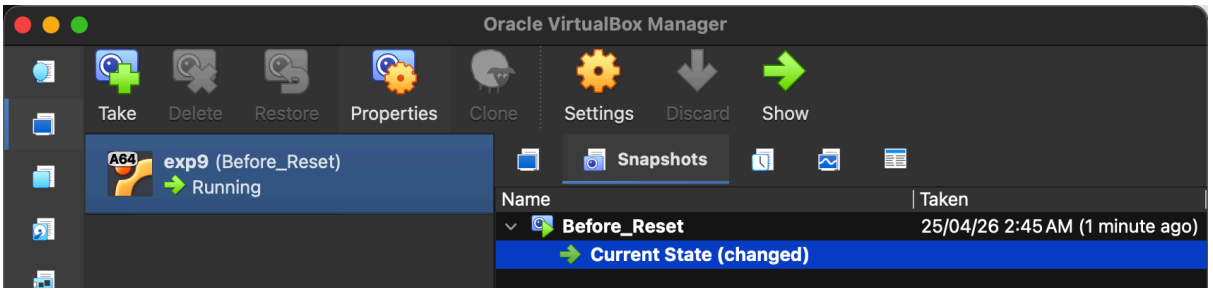
Step 2: Re-run instantly

```
[sheoraninfosec@Jigeshs-MacBook-Air exp9 % docker run -d -p 8082:80 exp9
 491361b59005772898be26de9c68a2ef318a9dd7fa2724c010849bb6c2229e48
 sheoraninfosec@Jigeshs-MacBook-Air exp9 %
```

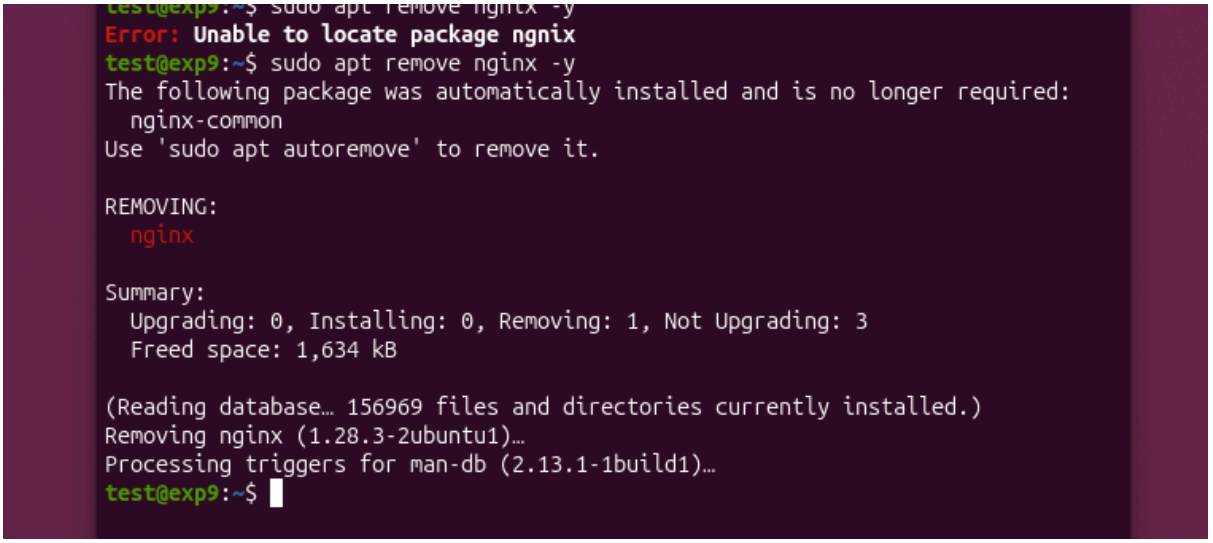
Task 5: Automate VM

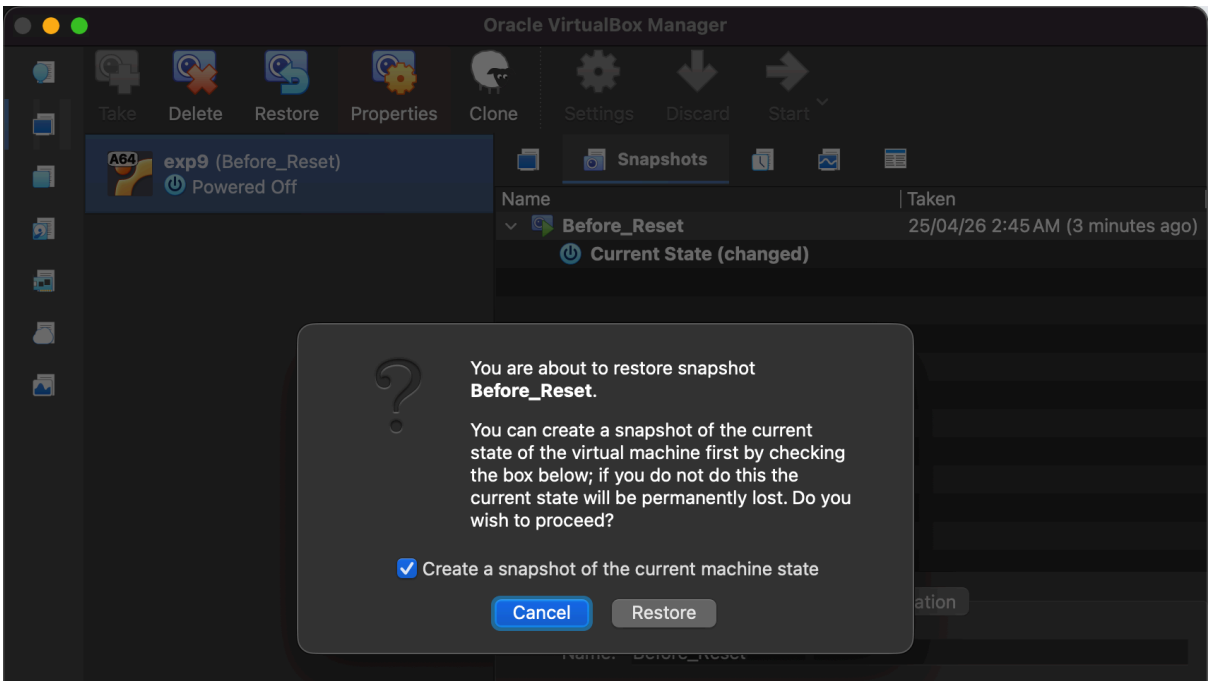
Prepare VM:

- Ubuntu is installed
- nginx is installed
- Your webpage is set



Break or change environment (for demo)





Observation:

- Snapshot allows restoring entire VM state instantly
- Eliminates need for manual reconfiguration
- Faster than reinstalling OS but slower than container redeployment

Task 6: Comparative Evaluation

Feature	Container (Docker)	Virtual Machine (VirtualBox)
Isolation Level	Process-level isolation (shared kernel)	OS-level isolation (separate OS)
Resource Utilization	Low (lightweight)	High (requires full OS resources)
Startup Time	Very fast (seconds)	Slow (minutes)
Deployment Speed	Instant (single command)	Slow (manual setup or snapshot restore)
Portability	High (runs anywhere with Docker)	Moderate (depends on VM configuration)

Task 7: Analytical Questions

How does Docker provide infrastructure isolation?

Docker provides isolation using:

- Namespaces → isolate processes, network, filesystem
- Control Groups (cgroups) → limit CPU, memory usage
- Containers run as isolated processes but share the host OS kernel

This ensures lightweight and efficient isolation.

How is isolation in VirtualBox different from containers?

- VirtualBox provides full OS-level isolation
- Each VM runs its own independent operating system
- Uses a hypervisor to emulate hardware

This results in stronger isolation but higher resource usage compared to containers.

Advantages and limitations of containerization over virtualization

Advantages:

- Lightweight
- Fast startup
- Efficient resource usage
- Easy deployment and scaling

Limitations:

- Less secure than VMs (shared kernel)
- Cannot run different OS kernels
- Limited isolation compared to VMs

How does automation differ between containers and VMs?

- Containers:
    - Automated using a single command (`docker run`)
    - Fast and repeatable
- Virtual Machines:
    - Require snapshots or cloning
    - Slower and more resource-intensive

Recommended approach for large-scale deployment

Containers are recommended
Reason:

- Faster deployment
- Better scalability
- Lower resource consumption
- Ideal for cloud and CI/CD environments

VMs are preferred only when:

- Strong isolation is required
- Different operating systems are needed

Conclusion

This experiment compared Docker containers and virtual machines in terms of isolation, performance, and automation. Containers were lightweight, fast, and easy to deploy, while virtual machines provided stronger isolation but required more resources and setup time.

Overall, containers are more suitable for modern DevOps and scalable deployments, whereas virtual machines are preferred when full system isolation is needed.

—----------END OF DOCUMENT —----------